

Low-Light Image Enhancement via Multi-Scale Feature Fusion

Software Project Lab 3 Technical Report

Submitted by

Soumitra Paul

BSSE 1317

Institute of Information Technology

University of Dhaka

Supervised by

Dr. Emon Kumar Dey

Associate Professor

Institute of Information Technology

University of Dhaka



Submission Date: 26 February, 2026

Letter of Transmittal

26 February 2026
BSSE 4th Year Exam Committee
Institute of Information Technology
University of Dhaka

Subject: Technical report on “*Low-Light Image Enhancement via Multi-Scale Feature Fusion.*”

Sir,

With due respect, I am pleased to submit the technical report of Software Project Lab 3 on “*Low-Light Image Enhancement via Multi-Scale Feature Fusion.*” In this report I have explained the details of my project design, implementation, and evaluation.

I extend my gratitude to Dr. Emon Kumar Dey for his continuous support throughout the project.

I tried my best to deliver a good understanding of the technical details of the project. However, it might lack perfection. So, I, therefore, hope that you will be kind enough to accept my report and oblige thereby.

Sincerely yours,
Soumitra Paul
BSSE 1317
Institute of Information Technology
University of Dhaka

Supervisor Signature

Acknowledgment

I am grateful to the Institute of Information Technology for giving me such a tremendous opportunity to work on “***Low-Light Image Enhancement via Multi-Scale Feature Fusion.***” I would like to convey my immense gratitude to my supervisor, Dr. Emon Kumar Dey, Associate Professor, Institute of Information Technology, University of Dhaka, for providing guidelines for preparing this analysis document. Lastly, I would like to thank my classmates.

They have always been helpful and provided valuable insights from time to time.

Abstract

This is a report on the software requirements, system architecture, and testing plan of the ***Low-Light Image Enhancement via Multi-Scale Feature Fusion***. This system is intended to improve the images taken under low light or even dark conditions where the images are normally characterized by a low level of brightness, high noise levels, poor contrast, and loss of visual details. These restrictions decrease the usefulness of images in human perception as well as computer-vision analysis.

To address these problems, the proposed tool uses deep learning models that rely on multi-scale feature fusion, in which the features of the images extracted at different scales are fused to maintain the global structure of the image as well as the finer details. This method makes the image brighter without making it look unnatural, and it keeps the texture and noise under control. The multi-scale fusion strategy also cuts down on overexposure and visual artifacts, which is not the case with older methods of enhancing images.

The system lets users post pictures taken in low light and see better results right away with an easy-to-use interface. A standard benchmark dataset for training and testing makes sure that the model is accurate, strong, and visually consistent. The main goal of this project is to make a stable and useful low-light image enhancement framework that can be used in real life in places like digital photography, surveillance systems, and image preprocessing pipelines.

Letter of Transmittal	2
Acknowledgment	3
Abstract	4
Chapter 1 : Introduction	6
1.1 Purpose	7
1.2 Scope	7
1.3 Assumptions	8
Chapter 2: Project Description	8
Chapter 2: Project Description	8
2.1 Quality Function Deployment	8
2.1.1 Normal Requirements	8
2.1.2 Expected Requirements	9
2.1.2 Excited Requirements	9
2.2 User Scenario	9
2.2.1 Data Input	9
2.2.2 Model Configuration	9
2.2.3 Image Enhancement	10
2.2.4 Visualization of Output Data	10
Chapter 3: Scenario Based Modeling	10
3.1 Use Case Diagram	10
3.1.1 Low Light Image Enhancement Tool	10
3.1.2 Low Light Image Enhancement Tool	11
3.1.3 Input Data Management	12
3.1.4 Model Configuration	13
3.1.5 Output Data Management	13
Chapter 4: Data Modeling	14
Chapter 5: Class-Based Modeling	15
Chapter 6: Architectural Design	22
6.1 Architectural Context Diagram	22
6.2 Defining Archetypes	23
6.3 Refining the Architecture into Top-Level Components	25
6.4 Instantiation of Software Architecture Components	26
Chapter 7: Applied Methodology	29
Chapter 8: User Manual and User Interface	35
Chapter 9: Test Report	42
9.1 High-Level Description of Testing Goals	42
9.2 Test Cases	43
Test Case 1: Valid Image Upload	43

Test Case 2: Invalid Input Handling	43
Test Case 3: Enhancement Output Accuracy	43
Test Case 4: UI Workflow and Batch Handling	44
Test Case 5: Performance Under Load	44
Test Case 6: Preference Level Adjustment	44
Chapter 10: Timeline	45
Chapter 11: Conclusion	46
References	48

Chapter 1: Introduction

Enhancing low-light images is an important part of computer vision and digital image processing. Pictures taken in low light or dark conditions usually have low visibility, low contrast, high noise, and loss of important visual details. These are a set of limits that make images less clear and hurt how people and automatic vision-based systems see things.

In other words, the Low-Light Image Enhancement Tool, which is based on the Multi-Scale Fusion Technique, is meant to be a useful and easy-to-use tool for making low-light images clearer and easier to see. The system can make things brighter, reduce noise, and keep colors and textures looking natural by using deep learning models that combine features from different resolution levels.

1.1 Purpose

The goals of this document are:

- Find and study the system requirements.
- Make a plan for the system's architecture
- Set up the testing plan
- Explain the method for making things better.
- Reduce overall development effort

1.2 Scope

The scope of the project is outlined below:

- An image enhancement application framework
- Enhancement of low-light images while preserving natural appearance
- Reduction of noise and improvement of texture and contrast

- Support for common image formats

1.3 Assumptions

The assumptions of the project are:

- Input images are captured in low-light conditions but retain basic visual information
- Uploaded images have acceptable resolution and format compatibility
- Users have basic knowledge of image quality and enhancement concepts

Chapter 2: Project Description

The chapter shows the quality function deployment and usage conditions of the proposed tool.

Quality Function Deployment: 2.1.

Quality Function Deployment (QFD) is a method that is applied to convert user requirements into technical specifications. In this project the following requirements have been discovered.

2.1.1 Normal Requirements

NR1: A framework

NR2: User is capable of uploading dark images.

NR3: There will be an improved output image that will be shown with the original image.

2.1.2 Expected Requirements

ER1: Easy and smart user interface.

ER2: A viable enhancement outcome that is aesthetically clear.

ER3: Image upload facilitated: File upload.

ER4: Multiple image formats (e.g., .jpg, .png) are supported.

ER5: Increase brightness of an image without affecting texture and color.

ER6: Rapid and high-performance image processing.

2.1.3 Excited Requirements

EXR1: The users can customize depending on the preference.

2.2 User Scenario

The low-light image enhancement system is intended to improve the quality of images taken in dark or low-light conditions and give an improved visual in accordance with analysis or display.

2.2.1 Data Input

The user is able to post low-light pictures in approved formats (like jpg or png). One can choose images either via a file browser in the local storage or a drag-and-drop system can be used for uploading images.

2.2.2 Model Configuration

It offers a choice of a pre-trained neural network model with the help of multi-scale fusion methods. This design gives an opportunity to incorporate better or different improvement models in the future.

2.2.3 Image Enhancement

The chosen model works with the uploaded image by removing and fusing multi-scale features. The process of enhancement increases brightness, removes noises, and restores the texture details without altering the natural look.

2.2.4 Presentation of the Output Data.

When it has been processed, the improved image is shown in the output window. Users are allowed to compare the original image in low light with the enhanced image.

Chapter 3: Scenario-Based Modeling

3.1 Use Case Diagram

Use case diagrams are given below.

3.1.1 Low Light Image Enhancement Tool

Level 0: Low Light Image Enhancement Tool

Primary Actors: User

Goal in Context: The diagram shown in Figure 1 represents the high-level overview of the Low Light Image Enhancement Tool System

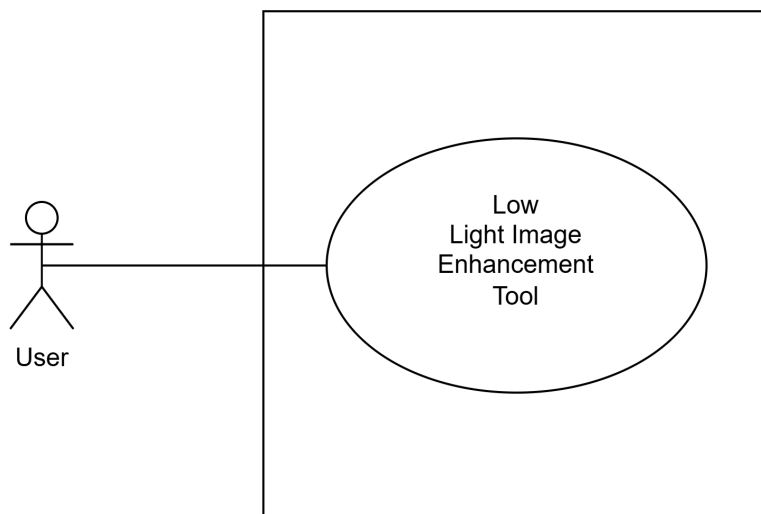


Figure 1: Level 0

3.1.2 Low Light Image Enhancement Tool

Level 1

Primary Actors: User

Goal in Context: The diagram shown in Figure 2 represents the modules of the low-light image enhancement tool. Modules of the projects are Data Input Management, Model Configuration, and Output Data Management.

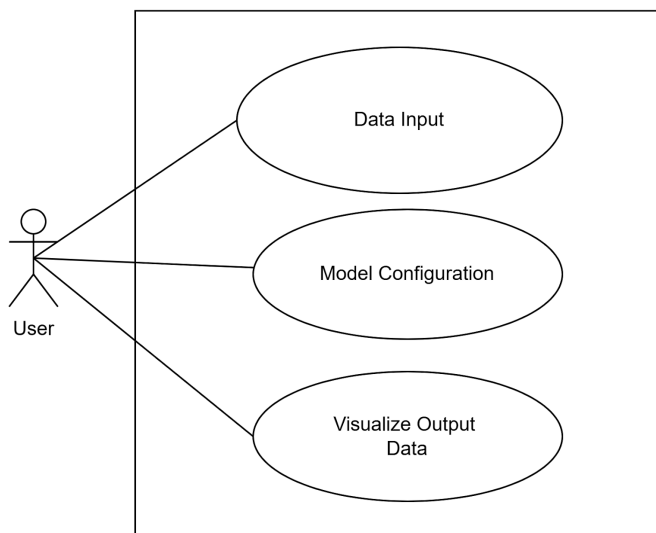


Figure 2: Modules of the low-light image enhancement tool use case

3.1.3 Input Data Management

Level 1.1: Input Data Management

Primary Actors: User

Goal in Context: The diagram in Figure 3 represents the use case for the Input Data Management Module. Users can load images, and the system will preprocess the loaded data.

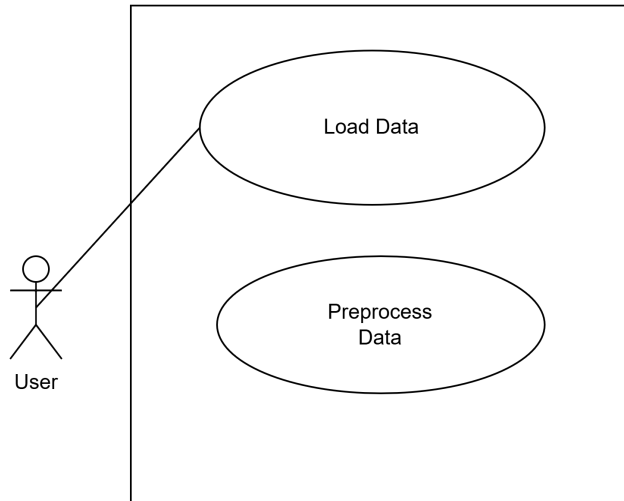


Figure 3: Input Data Management use case

3.1.4 Model Configuration

Level 1.3: Model Configuration

Primary Actors: User

Goal in Context: Users can load the pre-trained model. Moreover, the system will be responsible for training the model.

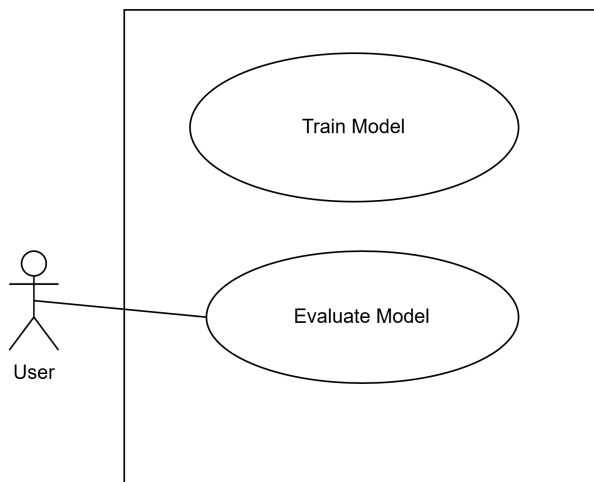


Figure 4: Model Configuration use case

3.1.5 Output Data Management

Level 1.4: Output Data Management

Primary Actors: User

Goal in Context: The diagram in Figure 5 shows the use case of the Output Data Management Module. Users can display and save the output.

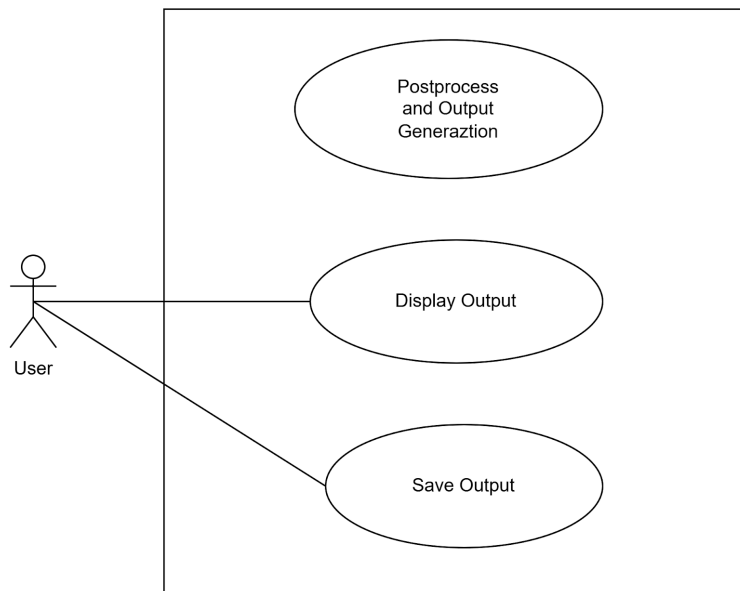


Figure 5: Output data management use case

Chapter 4: Data Modeling

The software development team will choose to design data models as part of the overall requirements modeling process if the software requirements call for the creation, extension, or interaction with a database or the construction and manipulation of complicated data structures.

The entity-relationship diagram (ERD) outlines each and every data object processed by the system, as well as the connections between them and details on how they are brought into the system, stored there, modified, and created. Our

trained model and its parameters are stored locally in the Low Light Image Enhancement Tool. In addition, for this project, we did not save any databases.

Chapter 5: Class-Based Modeling

This chapter describes the class-based modeling of the Low Light Image Enhancement Tool

5.1 Analysis Class

After identifying the nouns from the scenario, I have filtered the nouns belonging to the solution domain using general classification (external entities, things, events, roles, organizational units, places, and structures). Nouns selected as a potential class were filtered using selection criteria (retained information, needed services, multiple attributes, common attributes, common operations, and essential requirements). After performing an analysis on potential classes, I have found the following analysis classes:

1. Low-Light Image
2. WebApplication
3. User Preferences
4. EnhancementModel
5. ModelTrainer
6. Enhanced Image

5.2 Class cards

The class cards of these classes are shown below:

Low-Light Image	
Attributes	Methods

<input data-bbox="203 210 235 241" type="checkbox"/> input data <input data-bbox="203 252 235 283" type="checkbox"/> file_name <input data-bbox="203 294 235 325" type="checkbox"/> format	<input data-bbox="820 210 852 241" type="checkbox"/> getMetadata() <input data-bbox="820 294 852 325" type="checkbox"/> validateImage() <input data-bbox="820 378 852 409" type="checkbox"/> extractPixelData()
Responsibility	Collaborator
1. Represent an uploaded low-light image 2. Store image metadata required for processing	1. WebApplication 2. EnhancementModel

Table 1: class card of LowLightImage

WebApplication	
Attributes	Methods
<input data-bbox="203 1176 235 1207" type="checkbox"/> supportedFormats <input data-bbox="203 1218 235 1249" type="checkbox"/> activeSession <input data-bbox="203 1260 235 1291" type="checkbox"/> uiState	<input data-bbox="820 1176 852 1207" type="checkbox"/> acceptImageUpload() <input data-bbox="820 1260 852 1291" type="checkbox"/> collectPreferences() <input data-bbox="820 1344 852 1375" type="checkbox"/> startEnhancement() <input data-bbox="820 1428 852 1459" type="checkbox"/> displayEnhancedImage() <input data-bbox="820 1512 852 1543" type="checkbox"/> validateRequest() <input data-bbox="820 1596 852 1627" type="checkbox"/> handleError()
Responsibility	Collaborator
1. Provide a web-based interface for low-light	1. Low-Light Image

image enhancement 2. Accept image uploads and user preferences 3. Coordinate enhancement workflow and display results	2. User Preferences 3. EnhancementModel 4. Enhanced Image
---	---

Table 2: class card of WebApplication

User Preferences	
Attributes	Methods
- brightnessLevel - denoiseLevel - contrastLevel	+ updatePreferences() + resetToDefault() - validateRange()
Responsibility	Collaborator
1. Store and manage user-selected enhancement parameters	1. WebApplication 2. EnhancementModel

Table 3: class card of UserPreferences

EnhancementModel	
Attributes	Methods
- modelVersion - weightsPath - inputShape	+ loadModel() + enhanceImage(lowLightImage, userPreferences) + releaseResources() - preprocessInput() - postprocessOutput()
Responsibility	Collaborator
1. Apply multi-scale feature extraction and fusion 2. Enhance low-light images while preserving natural appearance 3. Perform model inference using trained parameters	1. WebApplication 2. Low-Light Image 3. User Preferences 4. ModelTrainer 5. Enhanced Image

Table 4: class card of EnhancementModel

ModelTrainer	
Attributes	Methods
- trainingConfig	+ trainModel()

- bestModelPath - evaluationMetrics	+ evaluateModel() + exportModel() - tuneHyperparameters()
Responsibility	Collaborator
1. Train the enhancement model using benchmark datasets 2. Evaluate model performance and select best weights 3. Export trained model parameters for deployment	1. EnhancementModel

Table 5: class card of ModelTrainer

Enhanced Image	
Attributes	Methods
- outputId - filePath - qualityMetrics	+ displaySideBySide() + exportImage() - computeMetrics()

Responsibility	Collaborator
1. Represent enhanced image output 2. Support visualization and comparison with original image 3. Enable export and download of enhanced results	1. WebApplication 2. EnhancementModel

Table 6: class card of EnhancedImage

5.3 Class Diagram

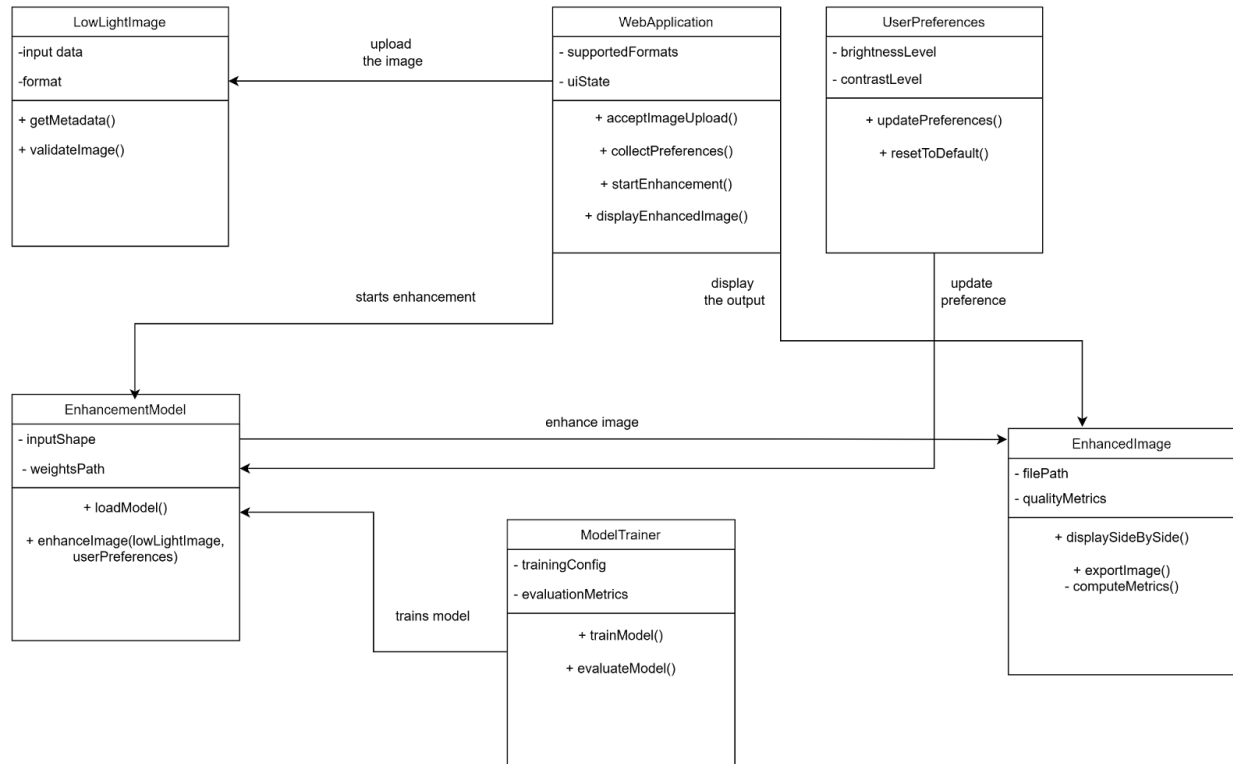


Figure 6: CRC class diagram of the low-light image enhancement tool

Chapter 6: Architectural Design

Architectural design is a visual representation in software engineering that outlines the high-level structure and organization of a software system. It focuses on defining the major components or modules of the system, their interactions, and the overall system's architecture.

6.1 Architectural Context Diagram

It is possible to model how software communicates with entities outside of its boundaries using an architectural context diagram. The target system's cooperating systems are denoted by the following symbols:

Superordinate systems:

Those systems that use the target system as part of some higher-level processing scheme. Our Low-Light Image Enhancement Tool has no superordinate systems, as it operates as a single application.

Subordinate systems:

Those systems that are used by the target system and provide data or processing necessary to complete the target system functionality. The Model Training System is the subordinate system of our Low-Light Image Enhancement Tool, as it provides the trained multi-scale fusion model used for image enhancement.

Peer-level systems:

Those systems that interact on a peer-to-peer basis, where information is produced or consumed by both the peer system and the target system. The system does not exchange data with external enhancement services or third-party platforms during runtime. There are no peer-level systems in this project.

Actors:

Actors are entities (people or devices) that interact with the target system by producing or consuming information required for processing. The User is the primary actor of our system, who uploads low-light images and receives enhanced

output images. Each external entity communicates with the target system through a web-based interface (the small shaded rectangles).

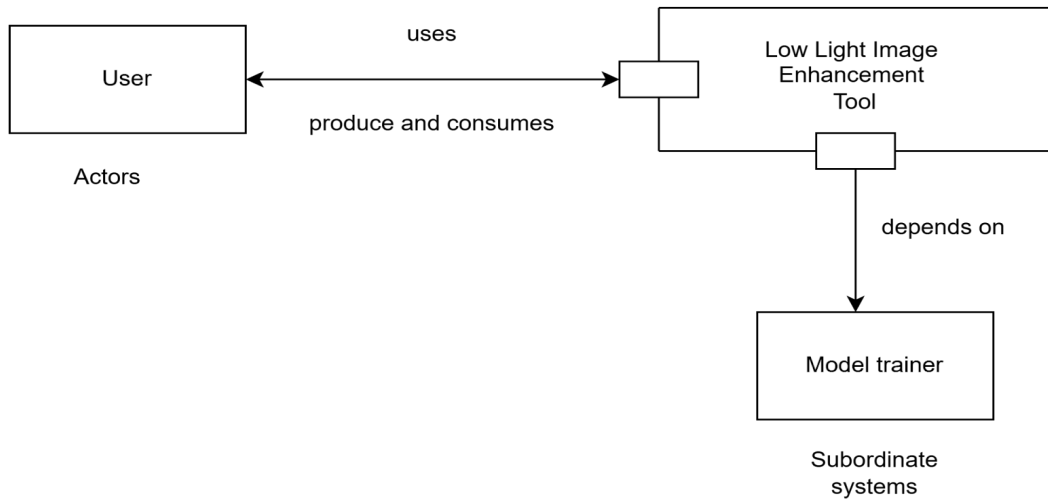


Figure 7: Architectural Context Diagram of the Low Light Image Enhancement Tool

6.2 Defining Archetypes

An archetype is a class or pattern that symbolizes a fundamental abstraction that is essential to the creation of the target system's architecture. In principle, even extremely complicated systems can be designed using a relatively small collection of archetypes. These archetypes, which stand for stable components of the goal system architecture but may be instantiated in several ways depending on the system behavior, make up the target system architecture.

Input:

Represents all input mechanisms, including user-uploaded low-light images and enhancement parameter selections.

Before processing: Before enhancement, it takes care of things like resizing, normalizing, and making adjustments that take noise into account.

Improvement (Multi-Scale Fusion Model): A core module that improves images taken in low light using a trained multi-scale fusion deep learning model.

Model Coach: An independent module that trains and tests the enhancement model with benchmark datasets and updates model parameters in shared storage.

Data set: Represents standard low-light image datasets used for training and testing.

Results: Shows output methods like better images, visual comparisons, and results that can be downloaded.

The system follows a **client–server architecture**, where the client manages input and visualization, and the server performs processing and model inference.

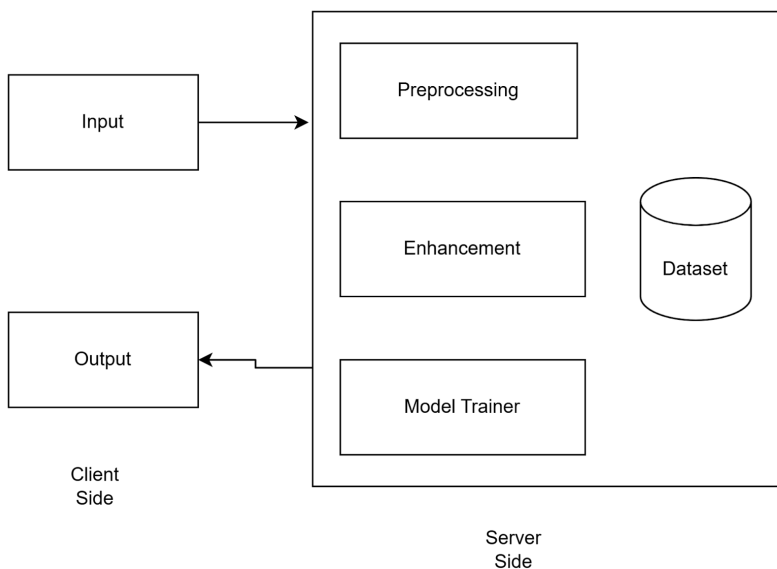


Figure 8: Identification of the Archetypes of the Low Light Image Enhancement Tool

6.3 Refining the Architecture into Top-Level Components

The system's structure starts to take shape as the software architecture is divided into its components. The Low Light Image Enhancement Tool's top-level parts are shown in the diagram below.

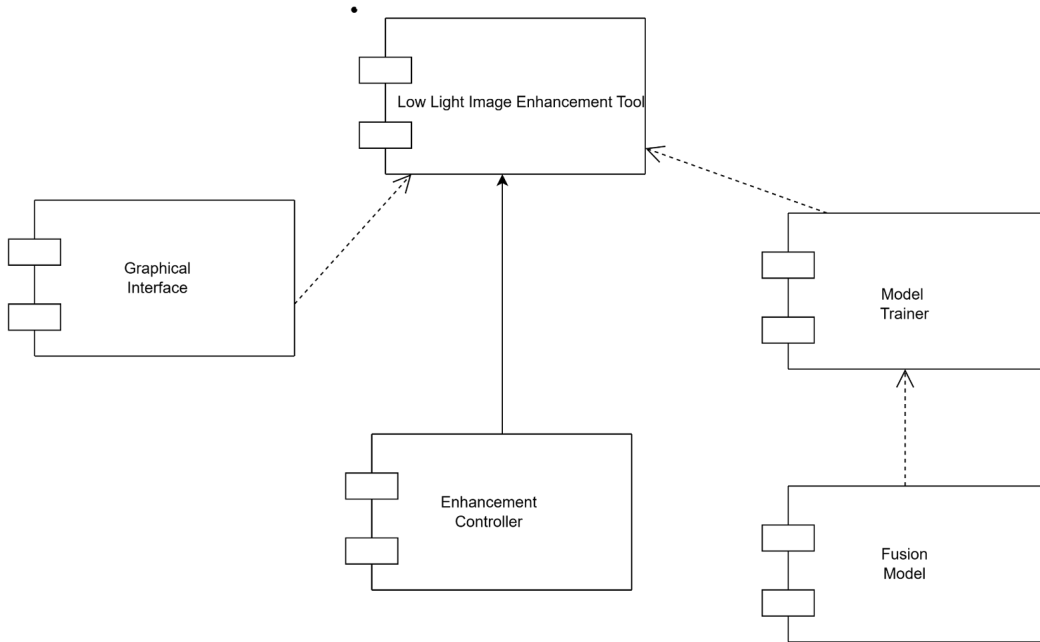


Figure 9: Top-level components of the Medical Image Segmentation Tool

Graphical Interface:

The Low-Light Image Enhancement Tool includes a web-based graphical user interface as a top-level component. It is responsible for handling image uploads, user interactions, parameter selection, and visualization of enhanced outputs.

Controller for Enhancements: This part is in charge of the workflow for improvements. It gets validated input images from the interface, controls the preprocessing steps, and sends the data to the enhancement model for inference.

The Multi-Scale Fusion Model: This is the main deep learning structure used to improve images taken in low light. To make images brighter and less noisy and

keep details, the model combines multi-scale feature extraction and fusion into a single end-to-end network.

Model Coach: This part is in charge of training and testing the multi-scale fusion model with benchmark datasets and changing the trained model parameters that the enhancement system uses.

6.4 Instantiation of Software Architecture Components

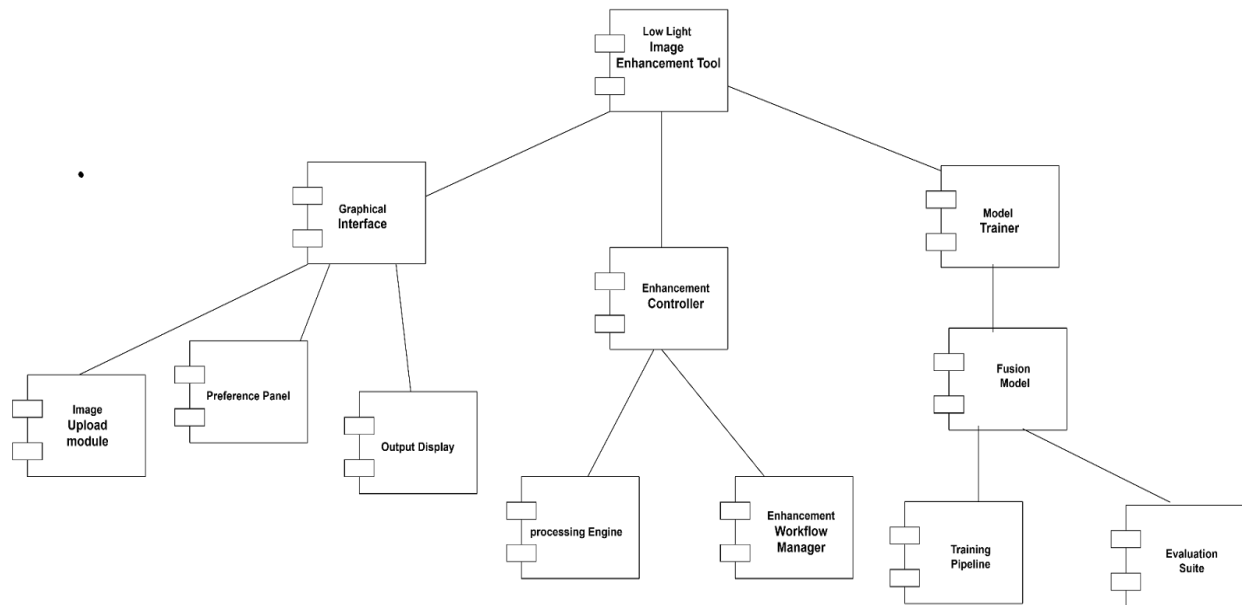


Figure 10: Instantiation of Software Architecture Components

6.5 Mapping Requirements for Software Architecture

Mapping Requirements to Components

Requirement	Mapped Component(s)
NR1: Web-based application	Graphical Interface
NR2: User can upload low-light images	Graphical Interface, Enhancement Controller
NR3: Enhanced output shown alongside original	Graphical Interface, Enhancement Controller
ER1: Simple and intuitive UI	Graphical Interface
ER2: Visually clear and natural enhancement results	Multi-Scale Fusion Model, Model Trainer
ER3: File upload & Drag & Drop support	Graphical Interface, Enhancement Controller
ER4: Support multiple image formats (.jpg, png)	Enhancement Controller, Graphical Interface
ER5: Enhance brightness while preserving texture & color	Multi-Scale Fusion Model, Model Trainer
ER6: Fast and efficient image processing	Enhancement Controller, Multi-Scale Fusion Model
EXR1: User-adjustable preferences	Graphical Interface, Enhancement Controller

Chapter 7: Applied Methodology

This chapter presents the theoretical foundations and mathematical modeling of the proposed Dual-Resolution Sparse Transformer (DRSformer) for low-light image enhancement. The methodology integrates classical illumination modeling with modern deep representation learning to address the intertwined challenges of illumination degradation, noise amplification, and texture loss.

Unlike heuristic enhancement techniques, the proposed approach formulates enhancement as a structured inverse problem solved through a learnable hierarchical transformation.

7.1 Workflow

The system is conceptualized through a high-level abstraction comprising three fundamental modules: input, processing, and output. This modular decomposition facilitates a clear comprehension of the system's functionality and interactions. The user interacts with the web interface to provide raw data, the backend performs the heavy computational restoration, and the output module renders the results for human evaluation.

$$\hat{I}_{final} = \mathcal{O}(\mathcal{P}(\mathcal{I}(I_{raw})))$$

Where:

- $\mathcal{I}$ = input preprocessing
- $\mathcal{P}$ = DRSformer processing
- $\mathcal{O}$ = output refinement

7.2 Input

The input module is responsible for managing user interactions and handling the ingestion of image files. It comprises the following components:

7.2.1 Web Interface & Interaction

The application employs a modern web-based frontend to secure user control. The implementation includes:

Intuitive Drag-and-Drop: Users can seamlessly upload images through a specialized drop zone that supports multiple file formats.

Request Interceptor: The frontend manages outgoing requests to the Flask server, including user-defined enhancement parameters like sharpness, gamma, and contrast.

State Management: The interface tracks the status of each image in the processing queue, providing real-time feedback.

7.2.2 Image Ingestion and Preprocessing

This sub-module handles the physical ingestion and mathematical preparation of the image data.

Supported Formats: The unit supports high-fidelity formats, including JPEG, PNG, BMP, and WebP, to ensure maximum compatibility.

Reflection Padding: To avoid boundary artifacts characteristic of convolutional layers, the system applies a mirrored padding strategy to ensure dimensions are divisible by the model's stride requirement.

Channel Normalization: Raw pixel values are transformed into a normalized distribution, allowing the neural networks to process subtle light variances with high precision.

Raw pixel values are transformed into a normalized distribution:

$$I_{norm} = \frac{I_{raw}}{255}$$

Where:

$$I_{norm} \in [0, 1]^{H \times W \times 3}$$

This normalization ensures numerical stability during deep network processing.

Validation Checking: The system verifies image integrity and size constraints before initiating the heavy inference pipeline.

7.3 Processing

The Processing module encompasses all backend operations that analyze the input image and generate the enhanced reflectance map. It is subdivided into four primary components:

7.3.1 Dual-Resolution Analysis

The system enhances images by processing them at different resolutions simultaneously:

Global Light Mapping: Lower resolutions focus on the overall scene illumination to ensure natural brightness distribution.

Local Texture Preservation: High-resolution branches focus on micro-details like edges and textures which are often hidden in dark shadows.

Frequency Interaction: The system merges these resolutions using cross-scale communication to ensure that sharpening doesn't introduce unnatural "halo" effects.

Lower-resolution representation:

$$F_{low} = \mathcal{D}(I_0)$$

High-resolution representation:

$$F_{high} = I_0$$

Where $\mathcal{D}(\cdot)$ denotes spatial downsampling.

Cross-scale interaction:

$$F = \mathcal{C}(F_{low}, F_{high})$$

Where $\mathcal{C}(\cdot)$ represents cross-resolution feature fusion.

7.3.2 Sparse Transformer Refinement

The core of the system utilizes advanced Transformer techniques to analyze the image content:

Feature Tokenization: The image is treated as a sequence of informative tokens rather than just flat pixels.

Sparse Attention Selection: Unlike standard models that process every pixel (including noise), the Sparse module selectively focuses on the most informative parts of the image, preventing the amplification of grain.

Long-Range Context: The attention mechanism allows the model to use light from one part of the image to logically "fill in" shadows in another.

Self-attention formulation:

$$\mathcal{A}(Q, K, V) = \text{Softmax} \left(\frac{QK^T}{\sqrt{d}} \right) V$$

Sparse attention selection introduces masking:

$$\mathcal{A}_{sparse}(Q, K, V) = \text{Softmax} \left(\mathcal{M} \odot \frac{QK^T}{\sqrt{d}} \right) V$$

Where mask $\mathcal{M}$ is defined as:

$$\mathcal{M}_{ij} = \begin{cases} 1, & \text{if } (QK^T)_{ij} \in \text{Top-}k \\ 0, & \text{otherwise} \end{cases}$$

7.3.3 Residual Feature Extraction

The restoration is governed by a Dilated Multi-Scale Residual Block (DMRB) approach:

Receptive Field Expansion: Dilated connections allow the model to capture a wide context of the scene without losing localized detail.

Residual Restoration: Instead of reconstructing the image from scratch, the model learns the "residual" (the light that is missing), which results in a much more authentic and stable final image.

7.3.4 Wavelet-Based Spatial Sampling

To ensure the integrity of the data, the system incorporates frequency-domain analysis:

Wavelet Decomposition: Uses the Discrete Wavelet Transform to split the image into high and low-frequency components.

Noise Suppression: Noise is primarily identified and suppressed in the high-frequency sub-bands, where it is most prevalent in low-light conditions.

Wavelet decomposition of input features:

$$\Phi_{DWT}(I_0) = \{I_{LL}, I_{LH}, I_{HL}, I_{HH}\}$$

Reconstruction using inverse wavelet transform:

$$I_{rec} = \Phi_{DWT}^{-1}(I_{LL}, I_{LH}, I_{HL}, I_{HH})$$

High-frequency components I_{LH}, I_{HL}, I_{HH} are primarily targeted for noise suppression.

7.4 Output

The output module manages the generation and dissemination of the enhanced results.

7.4.1 Result Visualization and Delivery

Base64 Encoding: For instantaneous web delivery, the restored image tensor is converted back into a Base64-encoded PNG string.

Download Management: A dedicated delivery sub-module allows users to save the high-resolution enhanced results directly to local storage.

Side-by-Side Comparison: The interface renders the original and enhanced versions in a result feed to allow for immediate qualitative evaluation.

7.4.2 User-Specific Post-Processing

Dynamic Refinement: Even after the AI finishes, the output module applies user-defined fine-tuning (Gamma and Contrast) to the final result, allowing for personalized aesthetic results.

Metadata Reporting: The output provides quantitative insights such as the processed resolution and completion status.

Gamma refinement:

$$\hat{I}_\gamma = \hat{I}^\gamma$$

Contrast adjustment:

$$\hat{I}_{final} = \alpha \hat{I}_\gamma + \beta$$

Where:

- γ : gamma correction factor
- α : contrast parameter
- β : brightness offset

7.5 Models and Their Significance

The project leverages a specialized Dual-Resolution Sparse Transformer (DRSformer), which represents a state-of-the-art approach to low-light tasks. Its significance lies in its ability to simultaneously perform denoising and illumination restoration through a single, unified pipeline, making it highly efficient for both real-time and high-fidelity applications.

The model is trained by minimizing a composite loss function:

$$\mathcal{L}_{total} = \lambda_1 \mathcal{L}_{char} + \lambda_2 \mathcal{L}_{ssim} + \lambda_3 \mathcal{L}_{color} + \lambda_4 \mathcal{L}_{tv}$$

7.6 Code Implementation Overview

app.py: Handles the Flask server logic, API endpoints, and communication with the frontend.

inference.py: Manages the high-level PyTorch inference lifecycle, including model loading and post-processing.

py: Contains the primary structural implementation of the DRSformer hierarchy.

mobilevit_block.py: Implements the Sparse Attention and Transformer components.

sampling.py: Executes the wavelet-based downsampling and upsampling logic.

Chapter 8: User Manual and User Interface

The Low-Light Image Enhancement System is an AI tool that can bring back the light in dark, poorly lit photos. It uses the Dual-Resolution Sparse Transformer (DRSformer) architecture to get rid of noise, fix lighting, and bring back texture detail in pictures taken in bad lighting.

8.1 An Overview of the System

Purpose: To enable users to restore low-light photographs using a state-of-the-art AI model, with customizable post-processing controls for a personalized visual result.

People who should use it:

Photographers and content creators who work in dark places.
Researchers assessing image restoration algorithms.
General users seeking to improve the quality of low-light photos.

Key Functionalities:

1. Automatic low-light image enhancement using DRSformer.
2. Batch processing of multiple images.
3. Processing a lot of images at once. essence, contrast, gamma,
4. Side-by-side comparison of original and enhanced images.
5. Download of enhanced results in PNG format.

8.2 User Manual

8.2.1 System Access

The system requires no account creation. Simply run the application locally and access it via a web browser.

How to Get There:How to Get There:

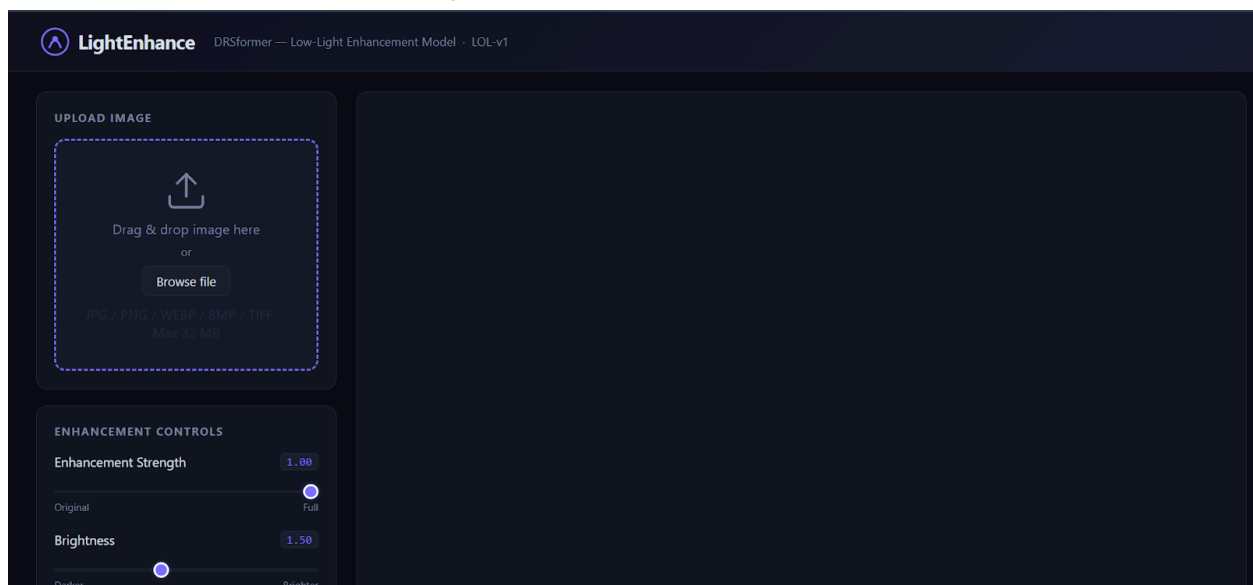
Launch a terminal in the project's folder. Launch a terminal in the project's
To run the app, type `python app.py`.To run the app, type `python app.py`.
Run the application: `python app.py`
Open your browser and go to `http://localhost:50000`. Open your browser
and go to `http://localhost:50000`.

8.2.2 Application Interface Overview

Drop Zone: The main place where you put your dark pictures.

Control Panel: A sidebar with sliders that lets you change the settings for enhancements.

Results Feed: A live feed below that shows the original result next to the improved one on each image card.



8.2.3 Using the System

Step 1: Upload an Image

Navigate to the Drop Zone at the top of the page. You can either:

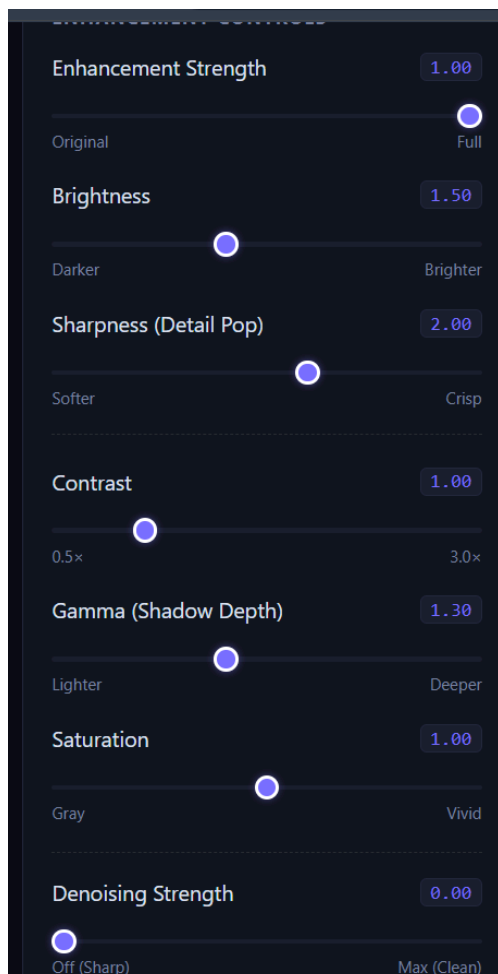
Drag and drop your image directly onto the zone.

To open a file browser, click on the zone.

Supported file types are JPG, PNG, BMP, and WebP. You can upload more than one picture at a time for batch processing.

Step 2: Adjust Enhancement Parameters

Before or after uploading, configure the enhancement using the sliders in the control panel:



Step 3: Run Enhancement

Press the "Enhance All" button. The system will:

Change the size of your image to the best resolution (600×400) automatically.

Load the DRSformer model (only the first request, which takes a few seconds).

Process each picture and show the result live in the Results Feed.

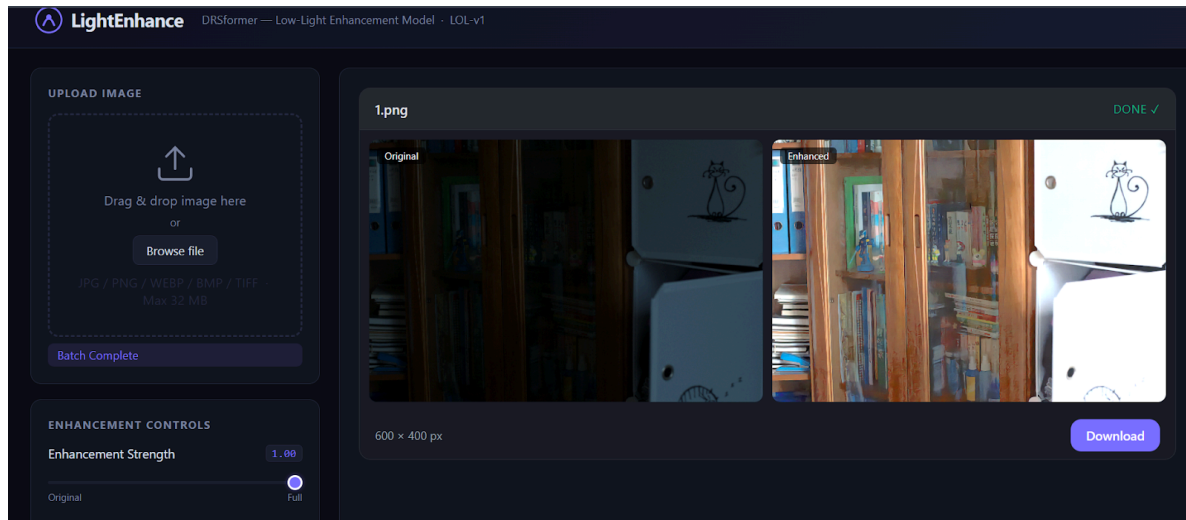
Set the status to Pending, Processing, and Done. ✓.

```
* Running on http://10.100.201.105:10000
Press CTRL+C to quit
* Restarting with watchdog (windowsapi)
* Debugger is active!
* Debugger PIN: 412-672-175
10.100.201.105 - - [25/Feb/2026 14:44:02] "GET / HTTP/1.1" 200 -
10.100.201.105 - - [25/Feb/2026 14:44:03] "GET /static/js/app.js HTTP/1.1" 304 -
10.100.201.105 - - [25/Feb/2026 14:44:03] "----- HTTP/1.1" 304 -
--- Loading Model: checkpoints/best1.pth Open file in editor (ctrl + click)
[load_model] STEP 4.1: torch.load from checkpoints/best1.pth
[load_model] STEP 4.2: Instantiating DRSformer
[load_model] STEP 4.3: Loading state dict
[load_model] STEP 4.4: Model ready
--- Model Loaded Successfully ---
--- Resizing image from (600, 400) to (600, 400) ---
--- Enhancing image: file (600x400) ---
10.100.201.105 - - [25/Feb/2026 14:45:51] "POST /enhance HTTP/1.1" 200 -
--- Resizing image from (600, 400) to (600, 400) ---
--- Enhancing image: file (600x400) ---
```

Step 4: View and Compare Results

Each image card in the Results Feed shows:

1. Left Panel: The original dark image.
2. Right Panel: The AI-enhanced version.
3. Status Badge: Confirms completion or error.
4. Resolution Info: Displays the processed dimensions.

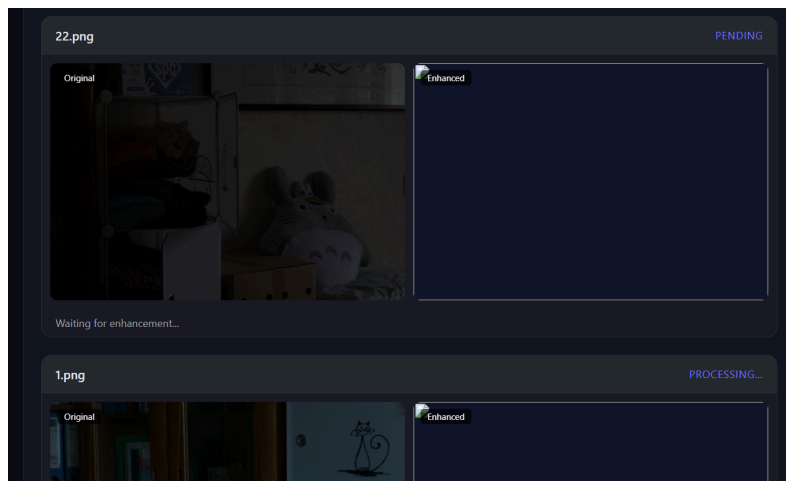
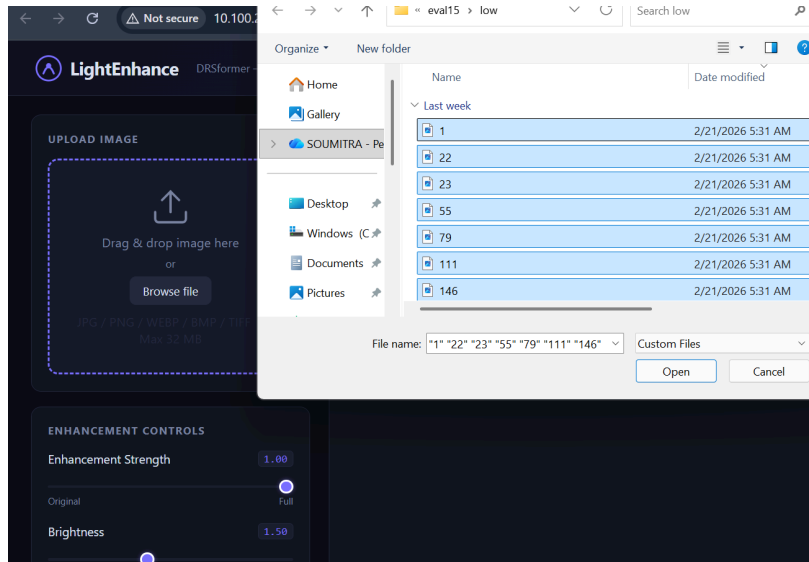


Step 5: Get the Results

After processing is done, each image card has a button that lets you download it. Click it to save the improved image as a PNG file on your computer.

8.3 Characteristics and Capabilities

1. **AI-Powered Enhancement:** The DRSformer architecture completely restores all lighting. Can deal with complicated shapes like faces, furniture, and scenes outside.
2. **Batch Processing** Upload and process a group of images all at once. The system puts them in order and shows how far along each one is.



3. Universal Image Input: Before processing, all images are automatically resized to a stable 600×400 resolution. This makes sure that performance is always the same, no matter what camera the original photo was taken with.

4. You can download enhanced results right away as high-quality PNG files with just one click.

8.4 Limitations

Device and Sensor Domain Gap

1. The model learned from the LOL-v1 and LOL-v2 datasets, which were taken with special laboratory cameras.
2. Smartphone pictures from the real world have different sensor noise profiles, lens optics, and color processing pipelines, which makes the domains not match.
3. As a result, images from modern phones may appear over-smoothed or exhibit color shifts after enhancement.
4. The model has trouble with real-world problems like optical distortion, motion blur, heavy fog, or haze, which weren't in the training data.
5. To make things better in the future, we need either domain adaptation or a training dataset that is more varied and works on more than one device.

8.5 Local Setup Instructions

To run the system on your local machine:

Step 1: Install Dependencies

```
pip install -r requirements.txt
```

Step 2: Place the Model Checkpoint

Ensure your trained model file (best1.pth) is located in:

```
low_light_enhancement/  
└── checkpoints/  
    └── best 1. path
```

Step 3: Run the application.

```
python app.py
```

Step 4: Access in Browser

Open `http://localhost:50000` in any modern web browser.

Chapter 9: Test Report

In this chapter, a high-level description of testing goals and a summary of features to be tested are presented.

9.1 A General Overview of Testing Goals

The system's image enhancement module must meet the following goals:

- To check that image ingestion is working, make sure that the system can correctly recognize and accept valid image formats (like PNG and JPEG) and reject files that don't work with them.
- To check enhancement algorithms: Make sure that the low-light enhancement processing makes the image brighter and clearer in a way that can be measured.
- To make sure the UI/UX works well, test the whole process from uploading to previewing to downloading to make sure the user has a good experience.
- To assess system stability: Verify the tool's ability to handle batch processing (e.g., 10+ images) without crashes or significant latency.

- To check the system's stability, make sure the tool can handle batch processing (like 10 or more images) without crashing or having a lot of lag. are accurately reflected in the final output.

9.2 Cases for Testing

Test Case 1: Uploading a Valid Image

Test Scenario: A user uploads a supported low-light image for enhancement.

Steps:: A user uploads a low-light image that is supported by improvement.

1. Please upload a valid picture file, such as a .jpg or .png.
2. Look at the upload. What you can look forward to:
 - The system accepts the picture.
 - Result: Passed.

Test Case 2: Invalid Input Handling

Test Scenario: A user tries to upload a file type that the enhancement tool doesn't support.

Steps:

1. Try to upload a file that isn't valid, like a .txt file. What you can expect:
 - The system rejects the file.
 - The message says, "Format not supported." Result: Passed.

Test Case 3: Enhancement Output Accuracy

Test Scenario: The system works on an image with low light to make it easier to see. **Test Scenario:** The system works on an image with low light to make it easier to see.

Steps:Steps:

1. Send in a low-light picture that is real. Send in a low-light picture that is real.
2. Use the enhancement algorithm. Use the enhancement algorithm.
What you can expect: What you can expect:
 - The images that have been improved are shown. The brightness and detail are clearly better than in the original. Result: Passed.
 - Compared to the original, the brightness and detail are clearly better. Result: Passed.ents in brightness and detail compared to the original.
Result: Passed.

Test Case 4: UI Workflow and Batch Handling

Test Scenario: The user goes through the whole process of uploading multiple images.

Steps: images.

1. Upload multiple images. **Result:** Passed.

Test Case 5: Performance When Under Load

Test Scenario: The system's stability when processing a group of images at the same time.

Steps:

1. Upload ten pictures all at once. What you can expect:
 - The system can handle all images without crashing.
 - Latency is still within acceptable limits. Result: Passed.

Test Case 6: Preference Level Adjustment

Scenario: The user changes the intensity of the enhancement for the same image.

Steps:

1. Send an image.
2. Change the levels of enhancement (low, medium, or high).

3. Improvement of the run. What you can expect:

- The output changes based on the chosen preference, such as "subtle" or "strong" enhancement.
- Selected settings are saved for the session.

Result: Passed.

Chapter 10: Timeline

This chapter talks about the project timeline, which has a detailed list of the main phases, tasks, and deadlines that will help the project go from start to finish. The main steps are:

- Gather Prerequisite Knowledge
- Documentation for the SRS Report
- Planning the whole project
- Growing
- Deployment of the final project
- Writing the Final Report

Task	Duration (Weeks)
Literature review	Weeks 1-2
Project selection	Weeks 2-3
SRS Documentation	Weeks 3-6
Designing	Weeks 3-6
Development	Weeks 6-11
Testing & Debugging	Weeks 11-13
Final Report Writing	Weeks 13-15
Submission	Weeks 15-16

Chapter 11: Conclusion

During the development of the image enhancement system, I have recorded the required technical specifications, which are essential in the development. The project scope, the assumptions, Quality Function

Deployment (QFD), and the user interaction scenarios are well discussed to make sure that the system could meet the requirements of low-light image optimization. Scenario-based modeling, data modeling, and class-based modeling have been used to define the structure and processing logic of the system clearly. In addition to the architectural design, this document demonstrates the important interrelations between the image processing engine, user interface, and the file management components.

The report has given a background description of the testing goals in a high level and a detailed overview of features and items that were put under rigorous testing. The fact that test cases have been included—valid/invalid uploads, performance under batch loads, and adjustments to the preferences of the user—confirms the fact that the system is stable and technically sound. Figures and tables used in this paper explain the technical intricacies of image histograms, improvement of images, and system lag time.

Although this system is a strong method of the strict analysis and enhancement of low-light image data, there is still much room to improve it. The scope in the future could involve the addition of more advanced deep learning models to reduce noise and the addition of super-resolution and real-time video enhancement features to give more adaptive and high-fidelity outputs.

This documentation provides a formal description of the system flow in which the reader would have a full picture of the technical design and functionality of the tool. This report is anticipated to be an excellent resource to stakeholders and image developers who have the responsibility of implementing this image enhancement technology and scaling it into the future.

References

- [1] Zhang, Hongxin, Teng Ran, Wendong Xiac, Kai Lv, Song Peng, Liang Yuan, and Jingchuan Wang. "Multi-Scale Progressive Fusion Network for Low-Light Image Enhancement." *IEEE Transactions on Instrumentation and Measurement* (2025).
- [2]Zhang, Kaibing, Cheng Yuan, Jie Li, Xinbo Gao, and Minqi Li. "Multi-branch and progressive network for low-light image enhancement." *IEEE Transactions on Image Processing* 32 (2023): 2295-2308.
- [3]<https://chatgpt.com>